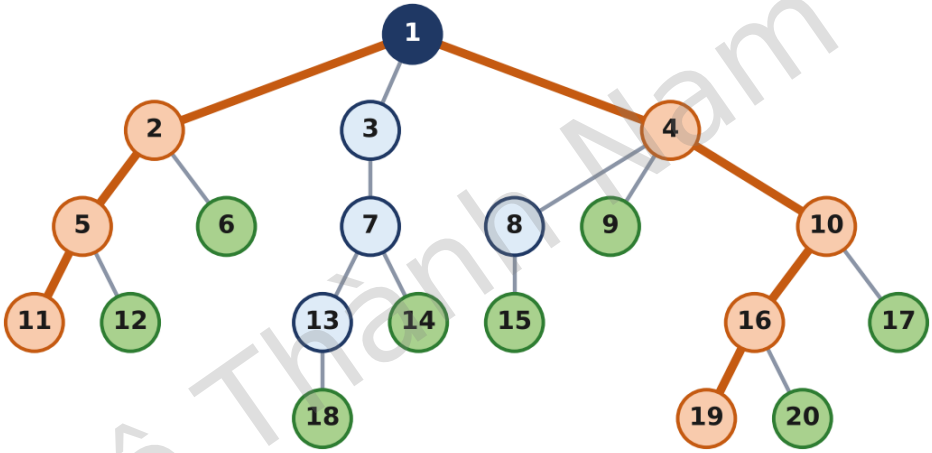
Tổng hợp và biên soạn: Lê Thành Nam

TÀI LIỆU BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

THUẬT TOÁN TRÊN CÂY

Dễ hiểu cho người bắt đầu từ con số 0

TẬP 1 · NỀN TẢNG



Lê Thành Nam

Biểu diễn · Duyệt cây · Euler tour · Đường kính · Tâm và trọng tâm

Chương 1. BIỂU DIỄN CÂY
Chương 2. DUYỆT CÂY
Chương 3. THỜI ĐIỂM VÀO – RA (EULER TOUR)
Chương 4. ĐƯỜNG KÍNH CÂY
Chương 5. TÂM VÀ TRỌNG TÂM

LỜI NÓI ĐẦU

Cây là cấu trúc dữ liệu xuất hiện nhiều nhất trong các đề thi học sinh giỏi Tin học sau mảng và đồ thị. Có đề hỏi trực tiếp về cây (đường kính, tâm, trọng tâm); có đề giấu cái cây bên trong một sơ đồ tổ chức, một mạng máy tính, một cây thư mục; và rất nhiều kĩ thuật nâng cao (tổ tiên chung gần nhất, chia cây thành chuỗi, chia để trị trên cây) đều xây trên vài viên gạch nền tảng giống nhau.

Khác với cách "đưa bài toán trước, rút ra lí thuyết sau", mỗi bài trong sách được chia làm hai phần rõ ràng:

• **Phần A – Lí thuyết:** trình bày thẳng vào kiến thức: định nghĩa, hình minh họa, thuật toán từng bước, chạy tay từng bước nhỏ (mỗi bước một hình), chứng minh từng ý (mỗi ý một hình), code mẫu, độ phức tạp, bẫy thường gặp.

• **Phần B – Bài vận dụng:** một bài toán theo khuôn đề thi, giải đầy đủ: ý tưởng ngây thơ có sơ đồ khối và chương trình vét cạn, chỉ ra chỗ bế tắc, áp dụng lí thuyết của Phần A để phá bế tắc, chạy tay, chương trình hoàn chỉnh, mở rộng và luyện tập.

Sách mở đầu bằng **Phần mở đầu: Cây và phương pháp giải tổng quát**, gồm toàn bộ thuật ngữ, bản đồ kiến thức của tập sách, và quy trình 6 bước để giải một bài toán trên cây. Tiếp theo là **12 bài học chia thành 5 chương**:

• **Chương 1. Biểu diễn cây:** cây là gì, ba tính chất nền tảng, danh sách kề, cây có gốc và mảng cha.

• **Chương 2. Duyệt cây:** BFS và độ sâu, DFS và kích thước cây con, DFS không đệ quy.

• **Chương 3. Thời điểm vào – ra (Euler tour):** kiểm tra tổ tiên trong O(1), duỗi cây thành mảng.

• **Chương 4. Đường kính cây:** hai lần BFS, quy hoạch động trên cây có trọng số.

• **Chương 5. Tâm và trọng tâm:** tâm của cây, trọng tâm của cây, ghép các cây với đường kính nhỏ nhất.

Các bài vận dụng gồm cả những bài kinh điển: **CSES Subordinates** (Bài 4), **CSES Tree Diameter** (Bài 8), **CSES Finding a Centroid** (Bài 11) và **IOI 2011 Dreaming** (Bài 12), được đặt lại trong bối cảnh nhà trường cho gần gũi. Mọi chương trình trong sách đều đã được kiểm tra bằng cách đối chiếu với chương trình vét cạn trên hàng trăm cây ngẫu nhiên, và chạy thử với cây $2 \cdot 10^5$ đỉnh (kể cả cây dạng "đường thẳng" sâu nhất).

Chương trình viết bằng C++17, đọc và ghi tệp theo kiểu "có tệp thì đọc tệp" để dùng được cả với phần mềm chấm Themis lẫn các trang chấm trực tuyến. Mọi chương trình đều không có chú thích bên trong; các dòng quan trọng được giải thích ngay bên dưới chương trình.

Chúc các em và các thầy cô học tập say mê, dạy tốt và đạt kết quả cao!

Lê Thành Nam

Lê Thành Nam

CÁCH HỌC VỚI TÀI LIỆU NÀY

Phần	Nội dung
Tóm tắt bài học	Khung tóm tắt ở đầu bài: kiến thức chính, khuôn cài đặt, độ phức tạp, khi nào dùng, bài vận dụng.
Phần A – Lí thuyết	Định nghĩa, tính chất, định lí kèm chứng minh từng ý; thuật toán; chạy tay từng bước nhỏ trên hình; code mẫu; độ phức tạp; bẫy; ghi nhớ.
Phần B – Bài vận dụng	Đề bài như đề thi thật: dữ liệu vào, kết quả, giới hạn, ví dụ.
B1. Ý tưởng ngây thơ	Cách làm ai cũng nghĩ ra: sơ đồ khối, chạy tay, chương trình vét cạn, đếm số phép tính.
Bế tắc	Chỉ ra chính xác vì sao cách ngây thơ quá chậm hoặc quá tốn bộ nhớ.
B2. Phá bế tắc	Áp dụng lí thuyết Phần A, từng bước, chạy tay trên hình.
B3. Chương trình hoàn chỉnh	Chương trình C++ đã kiểm chứng, giải thích các dòng quan trọng.
B4. Mở rộng, Luyện tập	Các biến thể thường gặp; bài luyện thêm và câu tự kiểm tra có đáp số.

Các khung màu trong sách có ý nghĩa cố định:

Khung	Ý nghĩa
TÓM TẮT BÀI HỌC	Tóm tắt toàn bộ bài ở ngay đầu bài.
ĐỊNH NGHĨA	Định nghĩa chính xác một khái niệm.
TÍNH CHẤT / ĐỊNH LÍ	Một khẳng định quan trọng, luôn có chứng minh đi kèm.
THUẬT TOÁN	Các bước của một thuật toán.
HỘP CÔNG CỤ C++	Giới thiệu một công cụ C++ khi nó được dùng lần đầu.
ĐẾM SỐ PHÉP TÍNH	Phân tích độ phức tạp: vì sao chậm, vì sao nhanh.
BẾ TẮC	Chỗ cách làm ngây thơ không vượt qua được.
BẪY THƯỜNG GẶP	Các lỗi hay gặp.
GHI NHỚ	Tóm tắt cuối phần lí thuyết.

Quy ước màu trong các hình được giữ nguyên trong cả cuốn sách: **đỉnh vàng viền đỏ** là đỉnh đang xét; **đỉnh xanh lá** là đỉnh đã xử lí xong (hoặc là lá, tùy hình); **đỉnh xanh dương nhạt** là đỉnh bình thường; **đỉnh viền nét đứt** là đỉnh chưa được thăm; **đỉnh tím** là đỉnh dùng trong chứng minh; **gốc** tô xanh đậm. Số nhỏ màu đỏ cạnh mỗi đỉnh là giá trị đang tính (độ sâu, kích thước, thời điểm, ...).

Lời khuyên khi học: Hãy đọc với giấy và bút bên cạnh. Gặp một hình chạy tay, hãy che phần lời giải lại, tự làm bước đó rồi mới đọc. Gặp một định lí, hãy tự vẽ một cái cây khác và kiểm tra định lí trên cây của mình. Chỉ khi tự viết lại được chương trình và tự giải thích được vì sao nó đúng mà không nhìn sách, em mới thật sự nắm được bài.

Lê Thành Nam

MỤC LỤC

LỜI NÓI ĐẦU **2**
CÁCH HỌC VỚI TÀI LIỆU NÀY **3**
PHẦN MỞ ĐẦU. CÂY VÀ PHƯƠNG PHÁP GIẢI TỔNG QUÁT **6**
CHƯƠNG 1. BIỂU DIỄN CÂY **12**
Bài 1. Cây và danh sách kề 13
Bài 2. Cây có gốc và mảng cha 36
CHƯƠNG 2. DUYỆT CÂY **53**
Bài 3. Duyệt theo chiều rộng (BFS) trên cây 54
Bài 4. Duyệt theo chiều sâu (DFS) và kích thước cây con 72
Bài 5. DFS không đệ quy, đếm lá và chiều cao 89
CHƯƠNG 3. THỜI ĐIỂM VÀO – RA (EULER TOUR) **105**
Bài 6. Thời điểm vào – ra (tin/tout) và quan hệ tổ tiên 106
Bài 7. Duỗi cây thành mảng 121
CHƯƠNG 4. ĐƯỜNG KÍNH CÂY **134**
Bài 8. Đường kính cây bằng hai lần BFS 135
Bài 9. Đường kính bằng quy hoạch động trên cây 149
CHƯƠNG 5. TÂM VÀ TRỌNG TÂM **165**
Bài 10. Tâm của cây 166
Bài 11. Trọng tâm của cây 179
Bài 12. Đường kính và tâm trên rừng 193
TỔNG KẾT: QUY TRÌNH GIẢI MỘT BÀI TOÁN TRÊN CÂY **205**

Lê Thành Nam

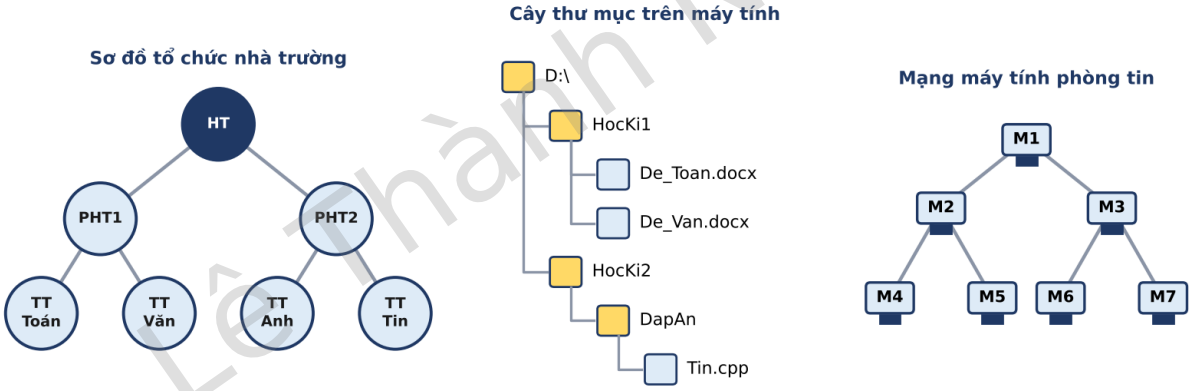
PHẦN MỞ ĐẦU

CÂY VÀ PHƯƠNG PHÁP GIẢI TỔNG QUÁT

Trước khi đi vào từng bài, hãy nhìn toàn cảnh. Phần này gom lại mọi thuật ngữ về cây mà cả cuốn sách dùng tới, vẽ bản đồ kiến thức của Tập 1, chỉ ra ý tưởng xuyên suốt "thông tin đi lên, thông tin đi xuống", và đưa ra quy trình 6 bước để giải một bài toán trên cây. Em có thể đọc lướt phần này lần đầu, rồi quay lại tra cứu mỗi khi gặp một từ lạ.

1. Cây ở quanh ta

Rất nhiều thứ quen thuộc trong nhà trường có hình dạng một cái cây: một "điểm gốc" tỏa ra các nhánh, mỗi nhánh lại tỏa tiếp, và **không có vòng kín**.



Hình 0.1. *Ba cái cây quen thuộc: sơ đồ tổ chức, cây thư mục, mạng máy tính*

Sơ đồ tổ chức có hiệu trưởng ở gốc; cây thư mục có ổ đĩa ở gốc; mạng máy tính thì không có gốc tự nhiên, ta có thể chọn bất kì máy nào làm gốc. Dù khác nhau về ý nghĩa, chúng có chung một cấu trúc toán học. Học thuật toán trên cây là học cách khai thác cấu trúc chung đó.

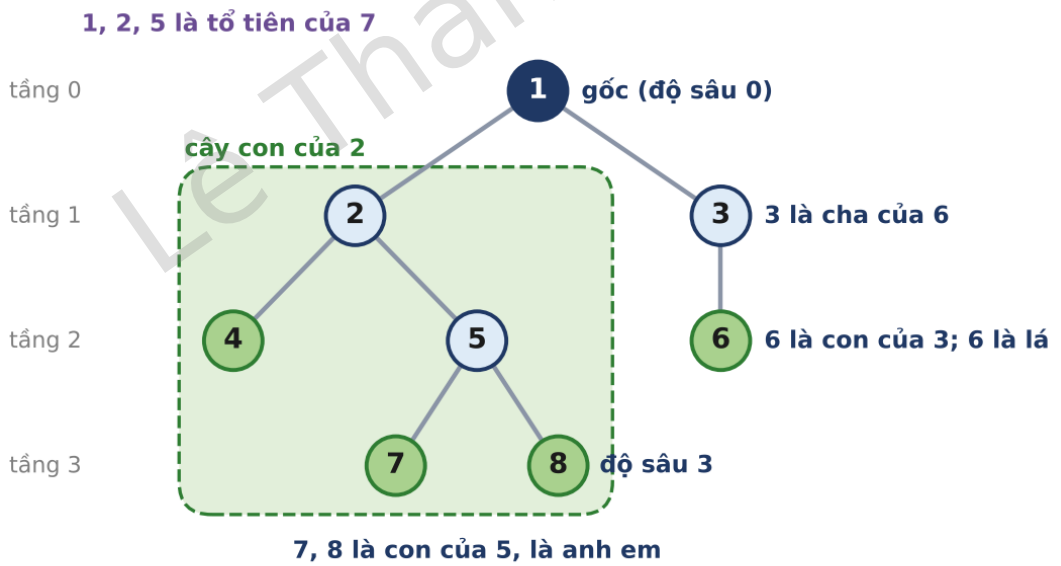
2. Những từ cần biết

Bảng dưới đây tập hợp mọi thuật ngữ dùng trong sách. Mỗi thuật ngữ sẽ được giới thiệu lại kĩ, có hình, ở bài học đầu tiên dùng tới nó.

Thuật ngữ	Ý nghĩa	Bài
Đỉnh, cạnh	Các "nút" và các "đoạn nối" giữa hai nút.	1
Cây	Đồ thị liên thông, không có chu trình; n đỉnh thì có n − 1 cạnh.	1
Bậc, lá	Bậc: số cạnh chạm vào đỉnh. Lá: đỉnh bậc 1.	1

Lê Thành Nam

Thuật ngữ	Ý nghĩa	Bài
Đường đi, khoảng cách	Giữa hai đỉnh có đúng một đường đi; khoảng cách là số cạnh (hoặc tổng độ dài cạnh) của nó.	1, 3
Gốc; cha, con; anh em	Chọn một đỉnh làm gốc. Đỉnh kề u và gần gốc hơn là cha của u; u là con của nó. Các con cùng cha là anh em.	2
Tổ tiên, con cháu	Các đỉnh trên đường từ u lên gốc (trừ u) là tổ tiên của u; u là con cháu của chúng.	2, 6
Độ sâu, tầng	Khoảng cách từ đỉnh tới gốc. Các đỉnh cùng độ sâu tạo thành một tầng.	2, 3
Cây con của u, sz[u]	u cùng mọi con cháu của u; sz[u] là số đỉnh của cây con đó.	4
Chiều cao	Khoảng cách xa nhất từ u xuống một lá trong cây con của u.	5
tin[u], tout[u]	Thời điểm vào u và số lớn nhất được đánh số trong cây con của u khi duyệt DFS.	6, 7
Đường kính	Khoảng cách lớn nhất giữa hai đỉnh của cây.	8, 9
Độ lệch tâm, bán kính, tâm	ecc(v): khoảng cách xa nhất từ v. Bán kính: ecc nhỏ nhất. Tâm: đỉnh đạt bán kính.	10, 12
Trọng tâm	Đỉnh mà khi bỏ đi, mọi mảnh còn lại có không quá n/2 đỉnh.	11
Rừng	Đồ thị không có chu trình (có thể không liên thông): gồm nhiều cây.	12

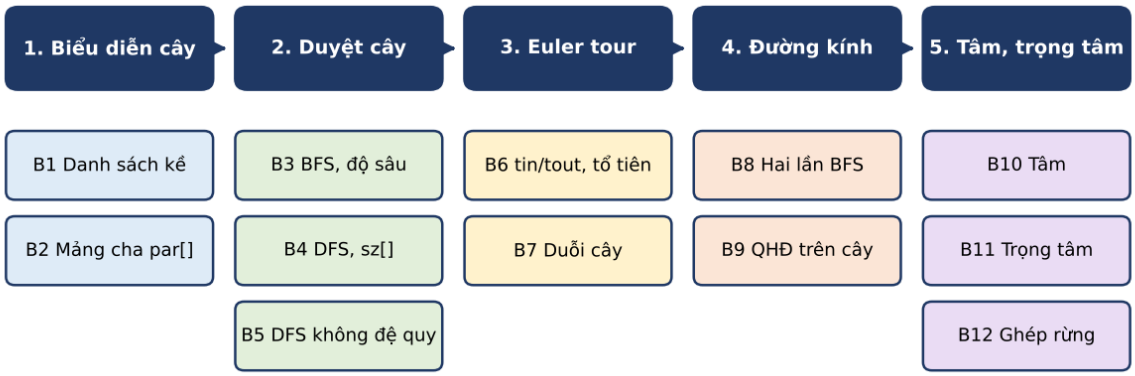


Hình 0.2. *Các thuật ngữ trên một cây có gốc tám đỉnh*

Cây tám đỉnh trên Hình 0.2 sẽ đi cùng em suốt Chương 1, 2, 3. Hãy nhìn kĩ: gốc là 1; tầng 1 có 2 và 3; lá là 4, 6, 7, 8; cây con của 2 gồm năm đỉnh 2, 4, 5, 7, 8.

Lê Thành Nam

3. Bản đồ kiến thức của Tập 1



Mỗi chương dùng lại công cụ của chương trước: danh sách kề → BFS/DFS → tin/tout → đường kính → tâm, trọng tâm

Hình 0.3. *Năm chương, mười hai bài: mỗi chương xây trên chương trước*

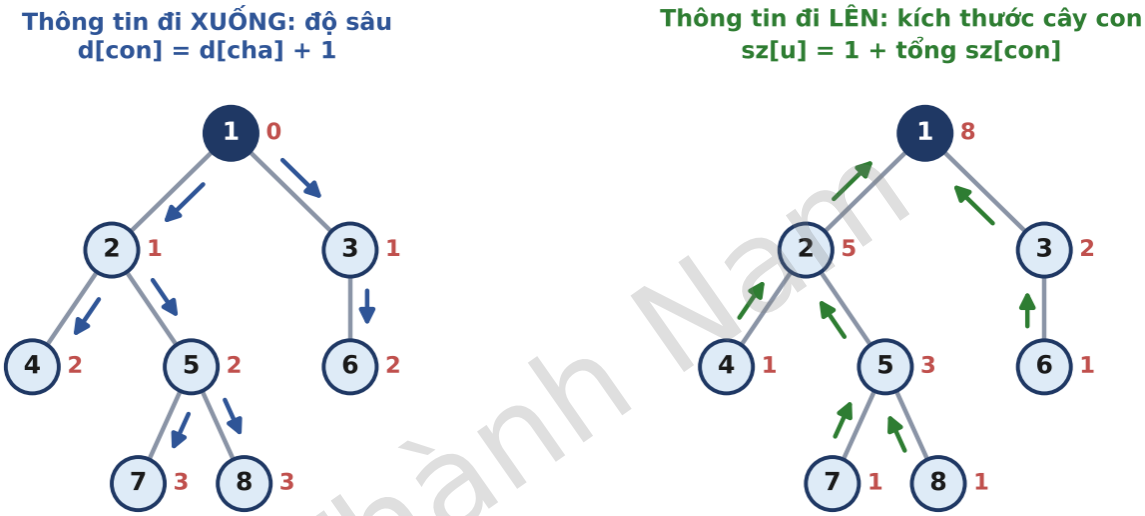
Bài	Kiến thức	Độ phức tạp	Bài vận dụng
1	**Cây và danh sách kề**	Bộ nhớ O(n); duyệt hàng xóm của mọi đỉnh O(n)	Mạng máy tính phòng tin học [MANGLAN]
2	**Cây có gốc và mảng cha**	O(n)	Đường báo cáo trong sơ đồ tổ chức [BAOCAO]
3	**Duyệt theo chiều rộng (BFS) trên cây**	O(n)	Truyền tin khẩn trong trường [TRUYENTIN]
4	**Duyệt theo chiều sâu (DFS) và kích thước cây con**	O(n)	Số cấp dưới (CSES Subordinates) [CAPDUOI]
5	**DFS không đệ quy, đếm lá và chiều cao**	O(n)	Tổ chuyên môn: số giáo viên đứng lớp và số cấp [LACAY]
6	**Thời điểm vào – ra (tin/tout) và quan hệ tổ tiên**	O(n) chuẩn bị, O(1) mỗi câu hỏi	Ai là cấp trên của ai? [CAPTREN]
7	**Duỗi cây thành mảng**	O(n log n) chuẩn bị, O(log n) mỗi câu hỏi	Thống kê giáo viên theo bộ môn [BOMON]
8	**Đường kính cây bằng hai lần BFS**	O(n)	Hai máy xa nhau nhất (CSES Tree Diameter) [XANHAT]
9	**Đường kính bằng quy hoạch động trên cây**	O(n)	Tuyến cáp dài nhất [DAYDAI]
10	**Tâm của cây**	O(n)	Chọn phòng đặt máy chủ [MAYCHU]
11	**Trọng tâm của cây**	O(n)	Chia trường thành các cụm (CSES Finding a Centroid) [CHIACUM]

Lê Thành Nam

Bài	Kiến thức	Độ phức tạp	Bài vận dụng
12	**Đường kính và tâm trên rừng**	O(n log n)	Nối mạng các điểm trường (IOI 2011 Dreaming) [DIEMTRUONG]

4. Ý tưởng xuyên suốt: thông tin đi lên, thông tin đi xuống

Phần lớn bài toán trên cây có dạng: **tính một con số cho mỗi đỉnh**. Khi đã chọn gốc, con số đó thường được tính theo một trong hai hướng:



Hình 0.4. *Hai hướng truyền thông tin trên cây có gốc*

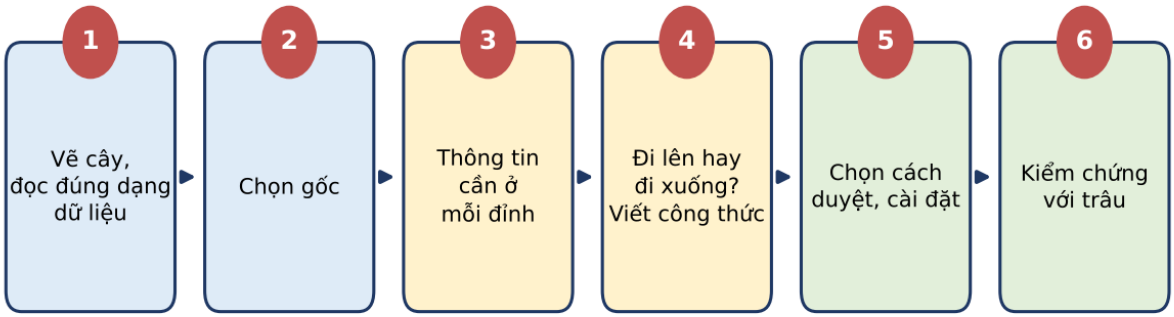
- **Đi xuống** (từ cha xuống con): con số của u tính được ngay khi biết con số của **cha** u. Ví dụ độ sâu: d[con] = d[cha] + 1. Phải tính cha **trước** con: duyệt theo thứ tự BFS, hoặc lúc **vào** đỉnh trong DFS.
- **Đi lên** (từ con lên cha): con số của u tính được khi đã biết con số của **mọi con** của u. Ví dụ kích thước cây con: sz[u] = 1 + tổng sz[con]. Phải tính con **trước** cha: duyệt thứ tự BFS **ngược**, hoặc lúc **ra** khỏi đỉnh trong DFS.

Đại lượng	Hướng	Công thức	Bài
Độ sâu d[u]	xuống	d[gốc] = 0; d[con] = d[cha] + 1	2, 3
Khoảng cách tới gốc (có trọng số)	xuống	dist[con] = dist[cha] + w(cha, con)	9
Kích thước cây con sz[u]	lên	sz[u] = 1 + Σ sz[con]	4
Số lá trong cây con	lên	la[u] = 1 nếu u là lá, ngược lại Σ la[con]	5
Chiều cao h[u]	lên	h[u] = 0 nếu là lá, ngược lại 1 + max h[con]	5
Nhánh xuống dài nhất dn[u]	lên	dn[u] = max (dn[con] + w)	9
tin[u] / tout[u]	theo thứ tự DFS	tin lúc vào, tout lúc ra	6, 7

Lê Thành Nam

5. Phương pháp giải tổng quát

Dù gặp bài nào, em đều có thể đi theo sáu bước sau. Mọi bài vận dụng trong sách đều được giải theo đúng sáu bước này.



Hình 0.5. *Sáu bước giải một bài toán trên cây*

Bước 1. Vẽ cây, đọc đúng dạng dữ liệu

Vẽ ví dụ của đề ra giấy. Xác định cây được cho dưới dạng nào: **danh sách n − 1 cạnh** (cây không gốc, lưu bằng danh sách kề, Bài 1) hay **mảng cha** "cấp trên trực tiếp của mỗi người" (cây có gốc, Bài 2). Đọc kĩ giới hạn: n tới $2 \cdot 10^5$ thì mọi cách $O(n^2)$ đều không dùng được.

Bước 2. Chọn gốc

Nếu cây chưa có gốc, hãy **chọn một gốc** (thường là đỉnh 1) và tính mảng cha par[] bằng một lần BFS (Bài 3). Có gốc rồi, mọi khái niệm cha, con, cây con, độ sâu mới có nghĩa. Nhiều bài không quan tâm gốc là đỉnh nào (đường kính, tâm, trọng tâm); khi đó gốc chỉ là "điểm tựa" để tính toán.

Bước 3. Xác định thông tin cần có ở mỗi đỉnh

Hỏi: "Nếu mỗi đỉnh u biết thêm **một con số** nào đó, câu trả lời có dễ không?". Ví dụ: muốn biết số cấp dưới của mỗi người, cần sz[u]; muốn biết u có là cấp trên của v, cần tin[] và tout[]; muốn biết đường đi dài nhất, cần nhánh xuống dài nhất dn[u].

Bước 4. Thông tin đi lên hay đi xuống? Viết công thức

Tra bảng ở Mục 4. Viết công thức tính con số của u từ con số của cha (đi xuống) hoặc của các con (đi lên). **Chạy tay** công thức trên ví dụ nhỏ trước khi viết code.

Bước 5. Chọn cách duyệt và cài đặt

Cách duyệt	Cho ta	Dùng khi
BFS từ gốc (Bài 3)	d[], par[], thứ tự ord[] (cha trước con)	thông tin đi xuống; khoảng cách; đặt gốc
Duyệt ord[] ngược (Bài 4)	con luôn xong trước cha	thông tin đi lên, cây có thể rất sâu
DFS đệ quy (Bài 4)	đúng thứ tự vào – ra, code ngắn	cây nông, n nhỏ (vài nghìn)

Lê Thành Nam

Cách duyệt	Cho ta	Dùng khi
DFS không đệ quy (Bài 5)	đúng thứ tự vào – ra, an toàn	cần tin/tout hoặc thứ tự DFS, n lớn

Mọi cách duyệt đều thăm mỗi đỉnh một lần và xét mỗi cạnh hai lần, nên đều chạy trong **O(n)**.

Bước 6. Kiểm chứng với chương trình trâu

Viết thêm một chương trình làm theo cách ngây thơ (chậm nhưng chắc chắn đúng), sinh hàng trăm cây ngẫu nhiên nhỏ và so sánh kết quả. Cách sinh một cây ngẫu nhiên n đỉnh rất đơn giản: với mỗi đỉnh i từ 2 đến n, chọn cha của nó là một đỉnh ngẫu nhiên trong 1..i − 1.

Sinh một cây ngẫu nhiên n đỉnh (dạng danh sách cạnh)

```
mt19937 rng(seed);
cout << n << '\n';
for (int i = 2; i <= n; i++) {
    int p = rng() % (i - 1) + 1;
    cout << i << ' ' << p << '\n';
}
```

Đừng quên thử thêm hai kiểu cây "khó": cây **đường thẳng** (1 – 2 – 3 – ... – n, sâu nhất, dễ làm tràn ngăn xếp đệ quy) và cây **hình sao** (mọi đỉnh nối vào đỉnh 1, một đỉnh có bậc n − 1).

TÓM TẮT PHẦN MỞ ĐẦU

- Cây: liên thông, không chu trình, n đỉnh n − 1 cạnh, giữa hai đỉnh có đúng một đường đi.
- Lưu cây bằng danh sách kề (cây không gốc) hoặc mảng cha (cây có gốc).
- Phần lớn bài toán trên cây là tính một con số cho mỗi đỉnh, truyền **đi xuống** (cha trước con) hoặc **đi lên** (con trước cha).
- Sáu bước: vẽ cây và đọc dữ liệu; chọn gốc; xác định thông tin; đi lên hay đi xuống; chọn cách duyệt và cài đặt; kiểm chứng.

Lê Thành Nam

CHƯƠNG 1. BIỂU DIỄN CÂY

Trước khi chạy được thuật toán, phải lưu được cái cây. Chương này trả lời hai câu hỏi: cây là gì, và lưu nó trong máy tính thế nào cho gọn và nhanh.

TRONG CHƯƠNG NÀY

Bài 1. Cây và danh sách kề *(vận dụng: mạng máy tính phòng tin học)*

Bài 2. Cây có gốc và mảng cha *(vận dụng: đường báo cáo trong sơ đồ tổ chức)*

Lê Thành Nam

BÀI 1 Cây và danh sách kề

Bài vận dụng: Mạng máy tính phòng tin học

TÓM TẮT BÀI 1. CÂY VÀ DANH SÁCH KỀ

Chủ đề: Biểu diễn cây

Kiến thức chính:

- Cây = đồ thị **liên thông, không chu trình**; n đỉnh thì có đúng **n − 1 cạnh**; giữa hai đỉnh có **đúng một** đường đi.
- Bậc, lá; cây có $n \geq 2$ đỉnh luôn có ít nhất hai lá.
- Ba cách lưu cây: danh sách cạnh, ma trận kề, **danh sách kề**.

Khuôn cài đặt:

```
vector<vector<int>> adj(n + 1);
for (mỗi cạnh u v) {
    adj[u].push_back(v);
    adj[v].push_back(u);
}
for (int v : adj[u]) ... ;
```

Độ phức tạp: Bộ nhớ O(n); duyệt hàng xóm của mọi đỉnh O(n)

Dùng khi: Mọi bài cho cây dưới dạng n − 1 cạnh

Bài vận dụng: Mạng máy tính phòng tin học [MANGLAN]

Lê Thành Nam

Học xong bài này, em sẽ:

- ✓ Hiểu chính xác **cây** là gì theo ngôn ngữ đồ thị: đỉnh, cạnh, đường đi, chu trình, liên thông.
- ✓ Nắm và **chứng minh** được ba tính chất nền tảng: cây có ít nhất hai lá, có đúng n − 1 cạnh, giữa hai đỉnh có đúng một đường đi.
- ✓ Biết ba cách lưu cây trong máy tính và vì sao **danh sách kề** là lựa chọn chuẩn.
- ✓ Tự tay xây danh sách kề từ danh sách cạnh và duyệt hàng xóm của mọi đỉnh trong O(n).

PHẦN A LÍ THUYẾT

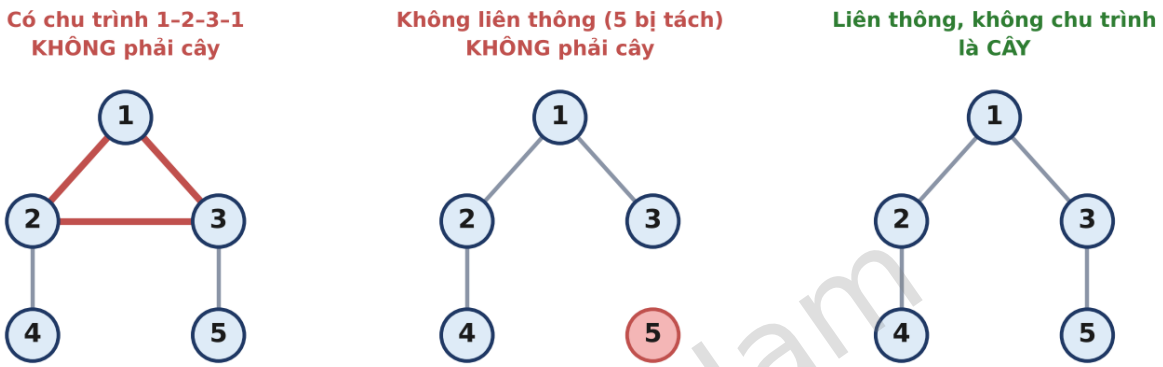
A1. Cây là gì?

Trước khi nói về cây, ta cần vài từ của **đồ thị**. Một đồ thị gồm các **đỉnh** (vẽ thành hình tròn, đánh số 1, 2, 3, ...) và các **cạnh** (vẽ thành đoạn thẳng nối hai đỉnh). Trong sách này các cạnh **không có chiều**: cạnh nối u với v cũng là cạnh nối v với u.

- **Đường đi** từ u tới v là một dãy đỉnh $u = x_1, x_2, ..., x_k = v$, trong đó hai đỉnh liên tiếp được nối bằng một cạnh, và không đỉnh nào lặp lại. **Độ dài** của đường đi là số cạnh của nó, tức k − 1.

- **Chu trình** là một đường đi quay trở về điểm xuất phát, dùng ít nhất ba đỉnh khác nhau, ví dụ 1 – 2 – 3 – 1.
- Đồ thị **liên thông** nếu giữa hai đỉnh bất kì luôn có ít nhất một đường đi.

> **ĐỊNH NGHĨA**
>
> **Cây** là một đồ thị **liên thông** và **không có chu trình**.



Hình 1.1. *Ba đồ thị năm đỉnh: chỉ đồ thị bên phải là cây*

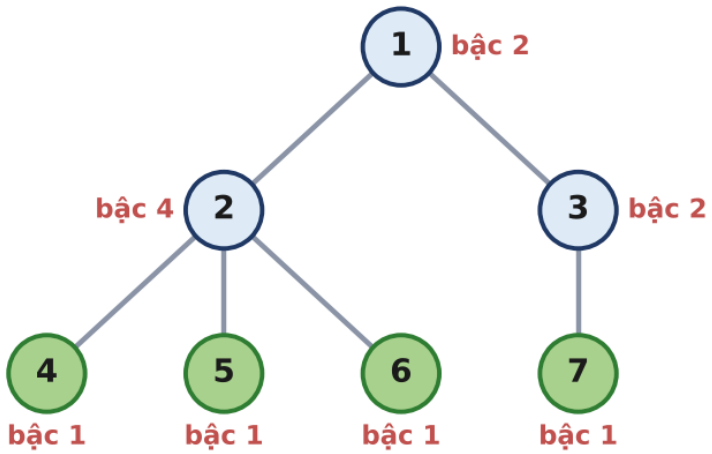
Trên Hình 1.1, đồ thị bên trái liên thông nhưng có chu trình 1 – 2 – 3 – 1 (tô đỏ); đồ thị ở giữa không có chu trình nhưng đỉnh 5 bị tách riêng, không đi tới được; chỉ đồ thị bên phải thỏa cả hai điều kiện nên mới là cây.

Cây xuất hiện ở khắp nơi trong đời sống nhà trường: sơ đồ tổ chức (hiệu trưởng, phó hiệu trưởng, tổ trưởng, giáo viên), cây thư mục trên máy tính, mạng máy tính phòng tin học nối bằng dây (không có vòng kín để tín hiệu khỏi chạy vòng), gia phả một dòng họ. Mọi bài toán trong cuốn sách này đều là bài toán trên những "cái cây" như vậy.

Lê Thành Nam

A2. Bậc và lá

> **ĐỊNH NGHĨA**
>
> **Bậc** của một đỉnh là số cạnh chạm vào đỉnh đó (cũng là số **hàng xóm** của nó).
>
> **Lá** là đỉnh có bậc bằng 1 (khi cây có từ hai đỉnh trở lên).



Hình 1.2. *Cây bảy đỉnh: bậc ghi cạnh mỗi đỉnh, bốn lá tô xanh*

Cây trên Hình 1.2 có 7 đỉnh và 6 cạnh. Đỉnh 2 có bậc 4 (nối với 1, 4, 5, 6); các đỉnh 4, 5, 6, 7 có bậc 1 nên là lá. Cộng bậc của mọi đỉnh: $2 + 4 + 2 + 1 + 1 + 1 + 1 = 12 = 2 \cdot 6$. Điều này luôn đúng, không chỉ riêng cây này:

> **TÍNH CHẤT TỔNG BẬC**
> Tổng bậc của mọi đỉnh bằng **hai lần số cạnh**, vì mỗi cạnh được đếm đúng hai lần: một lần ở mỗi đầu.

Tính chất nhỏ này sẽ cho ta một kết luận quan trọng ở Mục A5: duyệt hàng xóm của **tất cả** các đỉnh chỉ tốn khoảng $2(n - 1)$ bước.

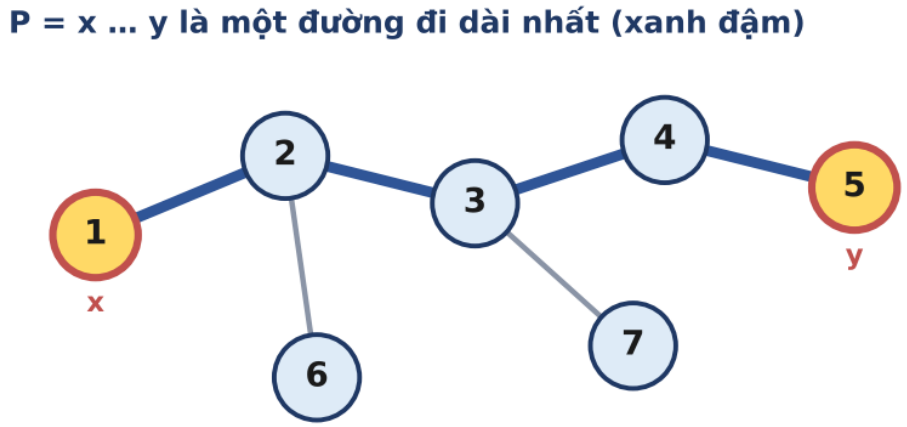
A3. Ba tính chất nền tảng của cây

Ba tính chất dưới đây được dùng ngầm trong mọi thuật toán về cây. Ta chứng minh từng tính chất, mỗi ý một hình.

Lê Thanh Nam

Tính chất 1: Cây có ít nhất hai đỉnh thì có ít nhất hai lá

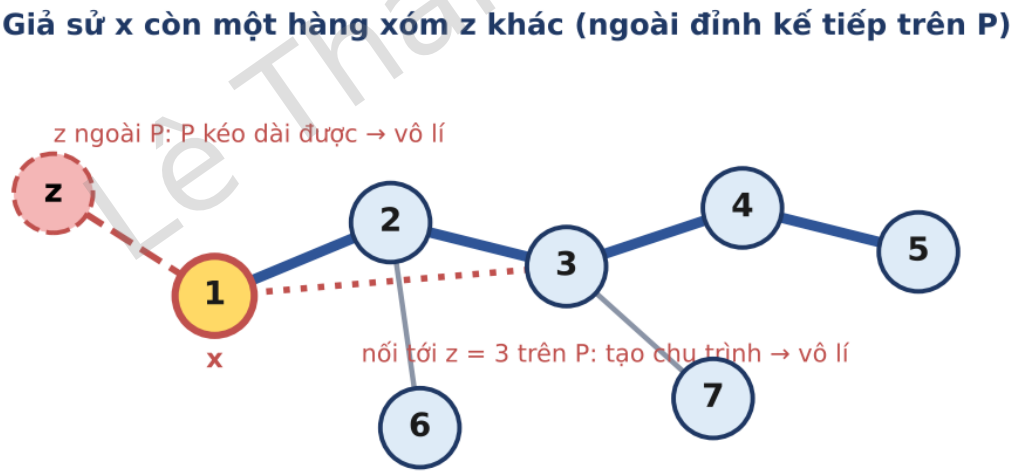
Ý 1 Lấy một đường đi dài nhất trong cây



Hình 1.3. *Lấy một đường đi dài nhất trong cây*

Cây chỉ có hữu hạn đường đi, nên có một đường đi **dài nhất** (nếu có nhiều đường dài bằng nhau thì lấy một đường bất kì). Gọi nó là P, hai đầu là x và y. Vì cây có ít nhất một cạnh nên $x \neq y$.

Ý 2 Giả sử đầu mút x có thêm một hàng xóm z



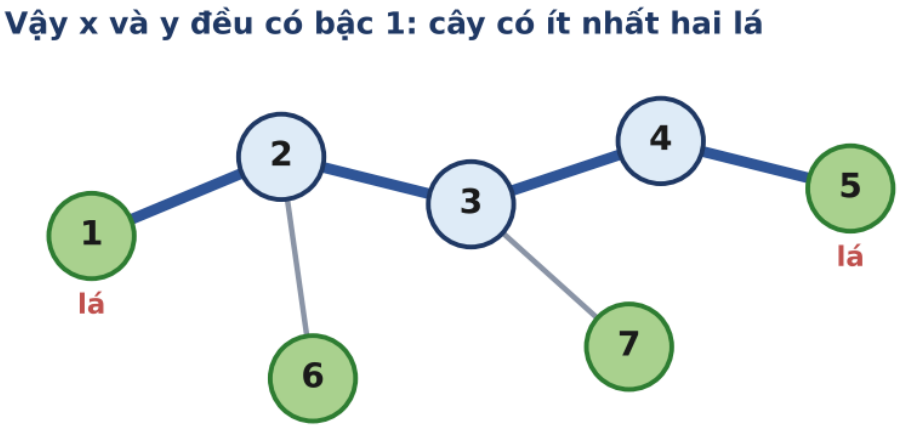
Hình 1.4. *Giả sử đầu mút x có thêm một hàng xóm z*

Ngoài đỉnh kế tiếp trên P, giả sử x còn nối với một đỉnh z khác. Có hai khả năng:

- z **không** nằm trên P: nối thêm cạnh z – x vào đầu P ta được đường đi dài hơn P. Trái với việc P dài nhất.
- z **nằm** trên P (ví dụ z = 3): đi từ x dọc P tới z rồi theo cạnh z – x quay về x, ta được một chu trình. Trái với việc cây không có chu trình.

Lê Thanh Nam

Ý 3 **Kết luận: hai đầu mút đều là lá**

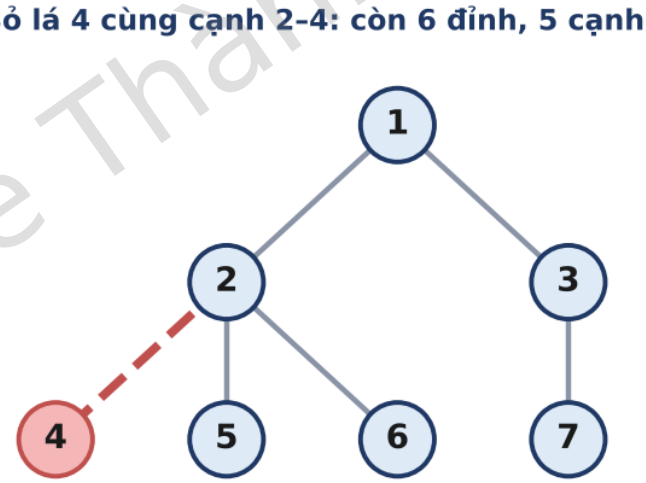


Hình 1.5. *Kết luận: hai đầu mút đều là lá*

Cả hai khả năng đều vô lí, nên x chỉ có đúng một hàng xóm: x là lá. Lập luận y hệt cho y. Vậy cây có **ít nhất hai lá** (cây trên hình thật ra có bốn lá).

Tính chất 2: Cây có n đỉnh thì có đúng n − 1 cạnh

Ý 1 **Bỏ một lá cùng cạnh của nó**

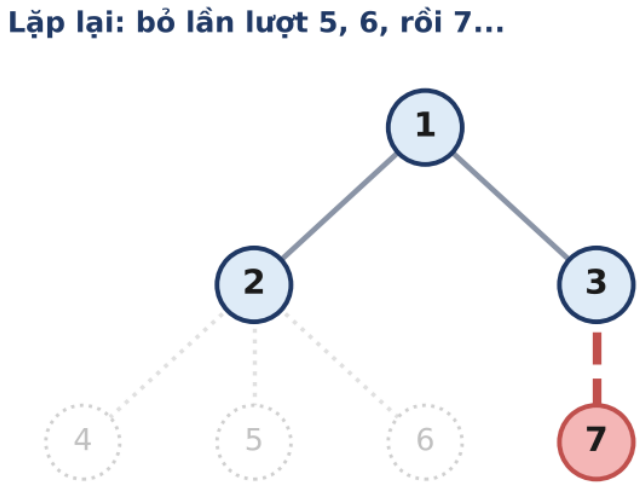


Hình 1.6. *Bỏ một lá cùng cạnh của nó*

Theo Tính chất 1, cây có một lá, chẳng hạn 4. Bỏ đỉnh 4 và cạnh duy nhất 2 – 4 của nó. Phần còn lại vẫn **liên thông** (không đường đi nào giữa hai đỉnh khác phải đi qua lá 4) và vẫn **không có chu trình**: vẫn là một cây, ít hơn một đỉnh và một cạnh.

Lê Thanh Nam

Ý 2 **Lặp lại cho cây mới**



Hình 1.7. *Lặp lại cho cây mới*

Cây mới lại có lá, lại bỏ được một đỉnh và một cạnh. Mỗi lần bỏ, số đỉnh và số cạnh cùng giảm đúng 1, nên **hiệu "số đỉnh − số cạnh" không đổi**.

Ý 3 **Khi chỉ còn một đỉnh**



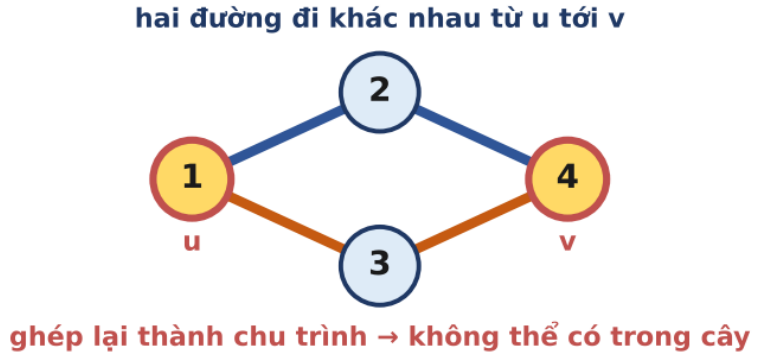
Hình 1.8. *Khi chỉ còn một đỉnh*

Cuối cùng còn 1 đỉnh và 0 cạnh: hiệu bằng 1. Vì hiệu không đổi trong suốt quá trình, lúc đầu cũng có số đỉnh − số cạnh = 1, tức là cây n đỉnh có **n − 1 cạnh**. Với cây 7 đỉnh: 6 cạnh.

Lê Thanh Nam

Tính chất 3: Giữa hai đỉnh bất kì có đúng một đường đi

Ý 1 **Có ít nhất một đường đi, và không thể có hai**



Hình 1.9. *Có ít nhất một đường đi, và không thể có hai*

Cây liên thông nên giữa u và v có ít nhất một đường đi. Nếu có hai đường đi khác nhau, đi theo đường thứ nhất từ u tới v rồi quay về u theo đường thứ hai, phần khác nhau của chúng tạo thành một chu trình (như hình, chu trình 1 – 2 – 4 – 3 – 1). Cây không có chu trình, nên đường đi là **duy nhất**.

> **BA CÁCH NÓI CÙNG MỘT ĐIỀU**
>
> Với một đồ thị n đỉnh, ba điều sau tương đương nhau (có một thì có cả ba):
>
> - liên thông và không có chu trình;
> - liên thông và có đúng n − 1 cạnh;
> - giữa hai đỉnh bất kì có đúng một đường đi.
>
> Vì vậy đề thi thường viết gọn: "n đỉnh, n − 1 cạnh, các đỉnh đôi một đi tới được nhau". Đó chính là một cây.

Tính chất 3 có ý nghĩa rất lớn cho lập trình: trên cây, **khoảng cách** giữa hai đỉnh chính là độ dài của **đường đi duy nhất** giữa chúng. Ta không cần thuật toán tìm đường ngắn nhất phức tạp; chỉ cần đi theo cây là được.

A4. Lưu cây trong máy tính

Đề bài thường cho cây dưới dạng **danh sách cạnh**: dòng đầu là n, sau đó n − 1 dòng, mỗi dòng hai số u v là một cạnh. Có ba cách lưu cây trong chương trình.

Lê Thành Nam

Cách 1: Giữ nguyên danh sách cạnh

n − 1 = 6 cạnh

1	2
1	3
2	4
2	5
2	6
3	7

Hình 1.10. *Danh sách 6 cạnh của cây bảy đỉnh*

Đơn giản nhưng rất bất tiện: muốn biết hàng xóm của đỉnh 2, ta phải đọc **cả danh sách** để tìm những dòng có số 2. Làm việc đó cho mọi đỉnh tốn n · (n − 1) bước.

Cách 2: Ma trận kề

	1	2	3	4	5	6	7
1	0	1	1	0	0	0	0
2	1	0	0	1	1	1	0
3	1	0	0	0	0	0	1
4	0	1	0	0	0	0	0
5	0	1	0	0	0	0	0
6	0	1	0	0	0	0	0
7	0	0	1	0	0	0	0

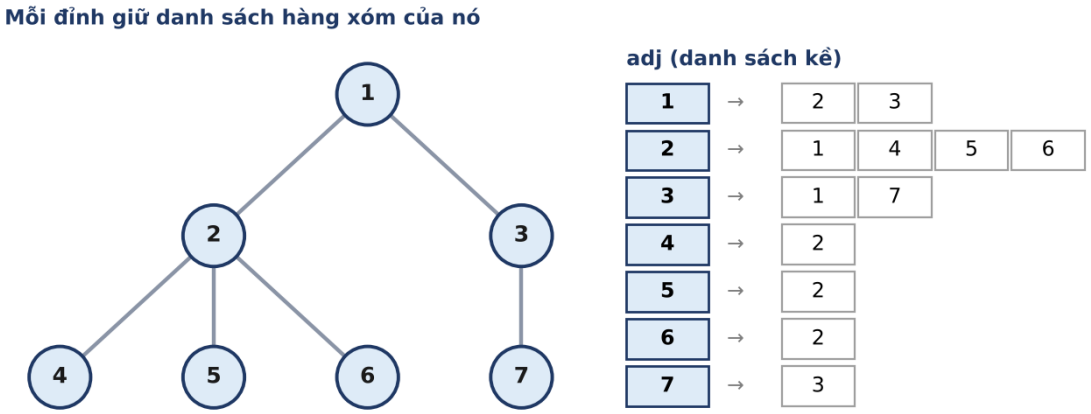
49 ô, chỉ 12 ô có số 1

Hình 1.11. *Ma trận kề: ô (u, v) bằng 1 khi có cạnh u – v*

Dùng bảng n × n, ô hàng u cột v bằng 1 nếu có cạnh u – v, ngược lại bằng 0. Kiểm tra hai đỉnh có nối nhau không chỉ mất một bước. Nhưng bảng có $\mathbf{n^2}$ ô trong khi chỉ 2(n − 1) ô có số 1. Với $n = 2 \cdot 10^5$: bảng có $4 \cdot 10^{10}$ ô, cần khoảng 40 GB bộ nhớ (mỗi ô 1 byte), gấp hàng trăm lần giới hạn của đề thi. Muốn liệt kê hàng xóm của một đỉnh vẫn phải đọc cả một hàng n ô.

Lê Thành Nam

Cách 3: Danh sách kề



Hình 1.12. *Danh sách kề: đỉnh u giữ một danh sách các hàng xóm của nó*

Mỗi đỉnh u giữ một danh sách **adj[u]** gồm đúng các hàng xóm của nó. Mỗi cạnh u – v xuất hiện hai lần: v trong adj[u] và u trong adj[v]. Tổng độ dài mọi danh sách là 2(n − 1): bộ nhớ chỉ cỡ n.

	Danh sách cạnh	Ma trận kề	Danh sách kề
Bộ nhớ	n − 1 cặp	n^2 ô	2(n − 1) số
Liệt kê hàng xóm của u	đọc cả danh sách: n	đọc cả hàng: n	**đúng bậc của u**
Hỏi "u, v có nối nhau?"	n	**1**	bậc của u
Dùng được với $n = 2 \cdot 10^5$?	được nhưng chậm	**không**	**được, và nhanh**

HỘP CÔNG CỤ C++: MẢNG CÁC VECTOR

`vector<vector<int>> adj(n + 1);` tạo n + 1 danh sách rỗng adj[0], adj[1], ..., adj[n] (bỏ không dùng adj[0] để đánh số đỉnh từ 1).

`adj[u].push_back(v);` thêm v vào cuối danh sách của u. `adj[u].size()` là độ dài danh sách, tức bậc của u.

`for (int v : adj[u])` lần lượt cho v nhận từng phần tử của adj[u]: duyệt mọi hàng xóm của u.

Code mẫu: Đọc cây vào danh sách kề

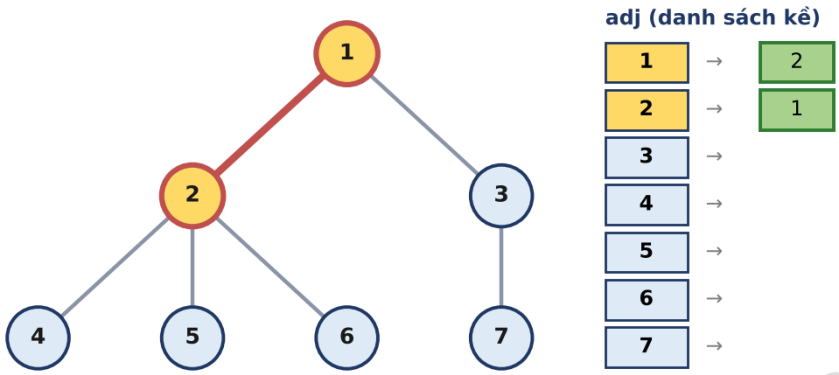
```
int n;
cin >> n;
vector<vector<int>> adj(n + 1);
for (int i = 0; i < n - 1; i++) {
    int u, v;
    cin >> u >> v;
    adj[u].push_back(v);
    adj[v].push_back(u);
}
```

Lê Thanh Nam

Chạy tay đoạn code trên với 6 cạnh của cây bảy đỉnh. Ở mỗi bước, cạnh vừa đọc tô đỏ, hai đỉnh của nó tô vàng, hai ô vừa được thêm tô xanh.

Bước nhỏ 1 **Đọc cạnh 1 2**

Đọc cạnh 1 2: thêm 2 vào adj[1], thêm 1 vào adj[2]

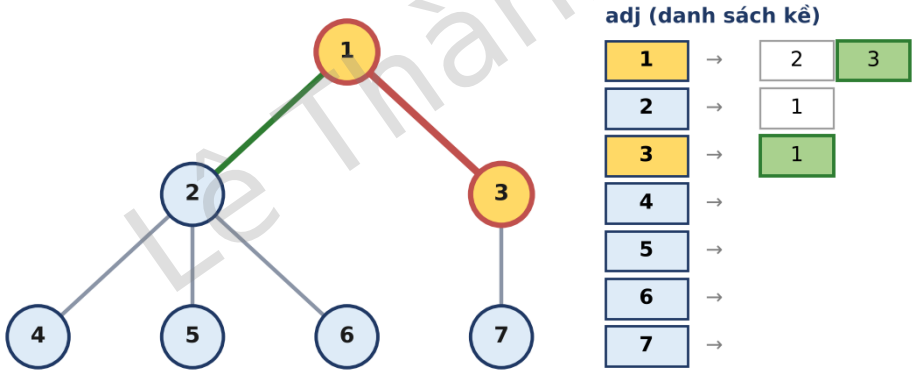


Hình 1.13. *Đọc cạnh 1 2*

adj[1] nhận 2, adj[2] nhận 1. Hai danh sách đầu tiên có một phần tử.

Bước nhỏ 2 **Đọc cạnh 1 3**

Đọc cạnh 1 3: thêm 3 vào adj[1], thêm 1 vào adj[3]



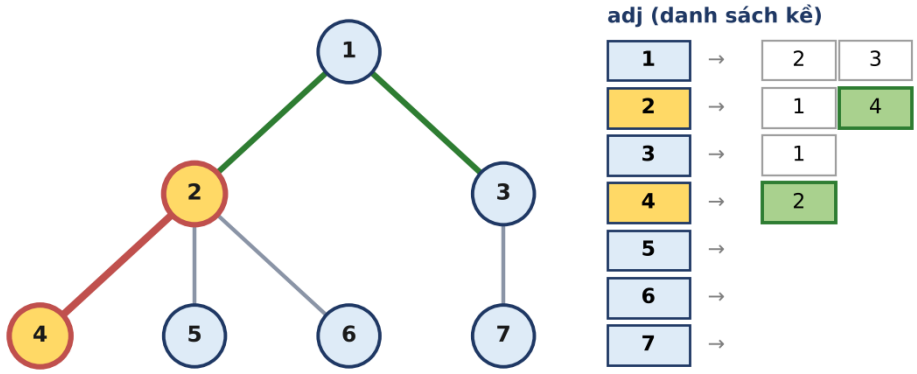
Hình 1.14. *Đọc cạnh 1 3*

adj[1] = {2, 3}; adj[3] = {1}.

Lê Thành Nam

Bước nhỏ 3 **Đọc cạnh 2 4**

Đọc cạnh 2 4: thêm 4 vào adj[2], thêm 2 vào adj[4]

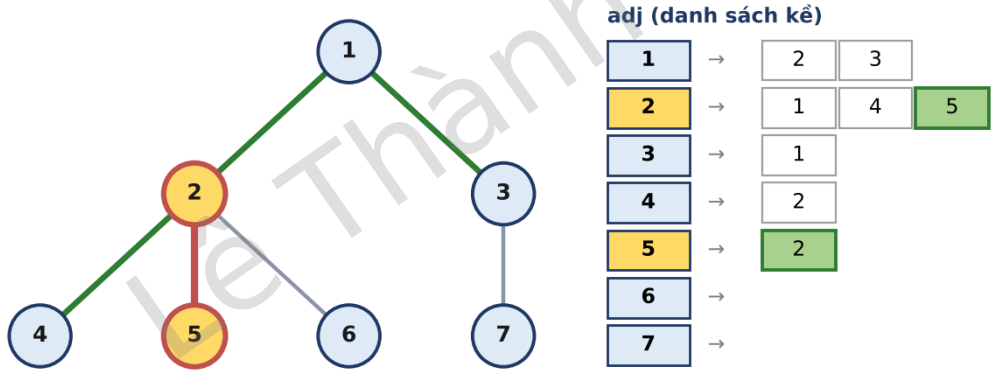


Hình 1.15. *Đọc cạnh 2 4*

adj[2] = {1, 4}; adj[4] = {2}. Để ý: số 4 được thêm vào **cuối** danh sách của 2, thứ tự hàng xóm chính là thứ tự đọc cạnh.

Bước nhỏ 4 **Đọc cạnh 2 5**

Đọc cạnh 2 5: thêm 5 vào adj[2], thêm 2 vào adj[5]



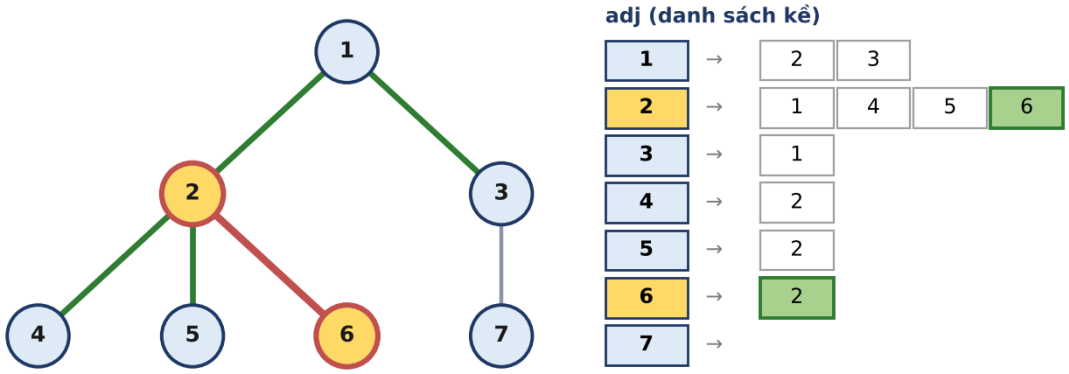
Hình 1.16. *Đọc cạnh 2 5*

adj[2] = {1, 4, 5}; adj[5] = {2}.

Lê Thành Nam

Bước nhỏ 5 **Đọc cạnh 2 6**

Đọc cạnh 2 6: thêm 6 vào adj[2], thêm 2 vào adj[6]

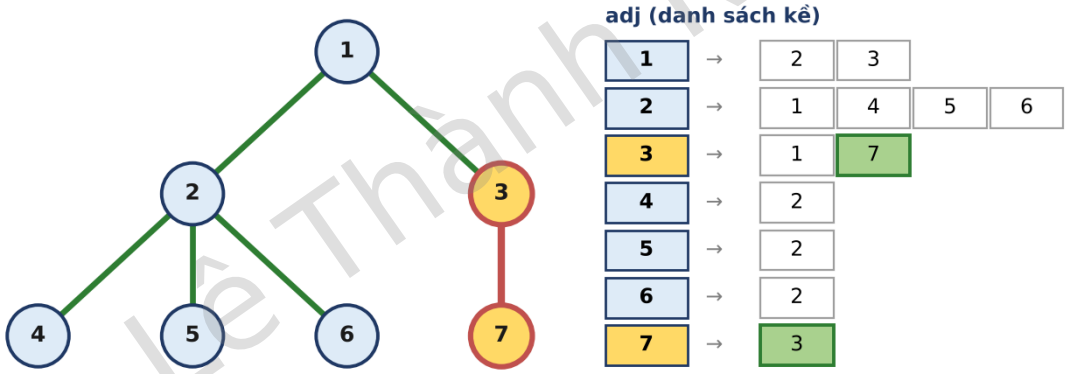


Hình 1.17. *Đọc cạnh 2 6*

adj[2] = {1, 4, 5, 6}; adj[6] = {2}.

Bước nhỏ 6 **Đọc cạnh 3 7**

Đọc cạnh 3 7: thêm 7 vào adj[3], thêm 3 vào adj[7]



Hình 1.18. *Đọc cạnh 3 7*

adj[3] = {1, 7}; adj[7] = {3}. Xong: tổng độ dài các danh sách là 2 + 4 + 2 + 1 + 1 + 1 + 1 = 12 = 2 · 6.

A5. Duyệt hàng xóm của mọi đỉnh

Code mẫu: Duyệt mọi cạnh qua danh sách kề

```
for (int u = 1; u <= n; u++)
    for (int v : adj[u]) {
        ...
    }
```

Đỉnh u có bao nhiêu hàng xóm thì vòng lặp trong chạy bấy nhiêu lần. Cộng lại cho mọi u, ta được **tổng bậc** = 2(n − 1) lần (Mục A2). Vậy hai vòng lặp lồng nhau ở trên chỉ tốn khoảng 2n bước, chứ không phải n^2 bước như hình dạng của nó gợi ý.

Lê Thành Nam

ĐẾM SỐ PHÉP TÍNH

- Đọc cây vào danh sách kề: O(n). Duyệt hàng xóm của mọi đỉnh: O(n).
- Với $n = 2 \cdot 10^5$: khoảng $4 \cdot 10^5$ lần thêm và $4 \cdot 10^5$ lần duyệt, chạy trong chớp mắt.

BẪY THƯỜNG GẶP

- Chỉ thêm một chiều `adj[u].push_back(v)`, quên `adj[v].push_back(u)`: v sẽ không "nhìn thấy" u.
- Khai báo mảng tĩnh `int a[200001][200001]`: vượt bộ nhớ, chương trình không chạy được.
- Vòng đọc cạnh chạy n lần thay vì $n - 1$ lần: đọc lố sang dữ liệu khác.
- Đề đánh số đỉnh từ 0 mà chương trình đánh số từ 1 (hoặc ngược lại).

GHI NHỚ

- Cây: liên thông, không chu trình; n đỉnh, $n - 1$ cạnh; giữa hai đỉnh có đúng một đường đi.
- Cây có $n \geq 2$ đỉnh luôn có ít nhất hai lá (đầu mút của đường đi dài nhất).
- Lưu cây bằng danh sách kề `vector<vector<int>> adj(n + 1)`, thêm cả hai chiều.
- Duyệt hàng xóm của mọi đỉnh tốn tổng bậc $= 2(n - 1)$ bước: O(n).

Lê Thành Nam